# RAJALAKSHMI ENGINEERING COLLEGE

THANDALAM, CHENNAI – 602105

## CS23432 / SOFTWARE CONSTRUCTION

(LAB ORIENTED THEORY COURSE)

Project Report on

**Veterinary Clinic Management System**
**(Spring PetClinic)**


Submitted by

**N. HARISH VIDYARTH**

Register No.: 2116241901035

III Year – B.E. CSE (Cyber Security)


*in partial fulfillment of the award of the degree of*

**BACHELOR OF ENGINEERING**

*in*

**COMPUTER SCIENCE AND ENGINEERING**
**(CYBER SECURITY)**

**ACADEMIC YEAR: 2026 – 2027**

**October 2026**

RAJALAKSHMI ENGINEERING COLLEGE


**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)**
**RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI**

# BONAFIDE CERTIFICATE

Certified that this project report **"VETERINARY CLINIC MANAGEMENT SYSTEM (SPRING PETCLINIC)"** is the bonafide work of **"N. HARISH VIDYARTH"** (Register No. 2116241901035), who carried out the project work for the course **CS23432 – Software Construction** under my supervision.

**SIGNATURE**

**Ms. Rupmala R**
Course Faculty, CS23432
Dept. of Computer Science and Engineering (Cyber Security)
Rajalakshmi Engineering College, Chennai

**Submitted for the Practical Examination held on _____**

**EXAMINER**

# CHAPTER 1 INTRODUCTION AND SOFTWARE SELECTION

## 1.1 Aim

To study, model and verify a well-engineered open-source application using the practices taught in CS23432: requirements, system modelling, design principles, testing and continuous integration.

## 1.2 Why Spring PetClinic

**Spring PetClinic** (`spring-projects/spring-petclinic`) is the Spring team's canonical reference application for a veterinary clinic. It was chosen because it is maintained by the framework authors, uses a clean layered MVC structure, has a real entity model, ships a broad automated test suite and a GitHub Actions workflow, and builds with a single command. All facts below were read from the cloned source at commit `500158f`.

| Item | Detail (from the repository) |
|---|---|
| Stack | Spring Boot 4.1.0, Java 17, Spring MVC, Spring Data JPA, Thymeleaf |
| Packages | `model`, `owner`, `vet`, `system` under `org.springframework.samples.petclinic` |
| Build | Maven (`pom.xml`) and Gradle; Maven wrapper included |
| CI | GitHub Actions: `maven-build.yml`, `gradle-build.yml`, `deploy-and-test-cluster.yml` |
| Tests | 19 test classes: 18 upstream plus `PetTests` added in this work |

**Table 1.1.** Project profile.

# CHAPTER 2 REQUIREMENTS

Requirements follow the Agile hierarchy of the course (theme → epic → story). The theme is *Run a veterinary clinic*; the stories below were derived from the controllers and views actually present in the application.

| ID | Epic | User story | Type |
|---|---|---|---|
| US1 | Owners | As a receptionist I can search owners by last name so I find a client quickly. | FR |
| US2 | Owners | As a receptionist I can register and edit an owner with address, city and telephone. | FR |
| US3 | Pets | As a receptionist I can add a pet with name, birth date and type to an owner. | FR |
| US4 | Visits | As a vet I can record a visit (date, description) against a pet. | FR |
| US5 | Vets | As a client I can view veterinarians and their specialties. | FR |
| US6 | Pets | As a vet I can see each pet's age in years on the owner page (added in this work). | FR |
| NF1 | Quality | Invalid input (blank name, bad telephone, future birth date) is rejected with a message. | NFR |
| NF2 | Delivery | Every change builds and passes all tests automatically in CI. | NFR |
| NF3 | Portability | The app runs on an in-memory database with no setup, and on MySQL/PostgreSQL. | NFR |

**Table 2.1.** Functional (FR) and non-functional (NFR) user stories.

# CHAPTER 3 SYSTEM MODELLING AND DESIGN

## 3.1 Class Model

The domain follows a small inheritance tree: `BaseEntity` (id) is extended by `Person` (first and last name) and `NamedEntity` (name). `Owner` and `Vet` specialise `Person`; `Pet`, `PetType` and `Specialty` specialise `NamedEntity`. An owner holds many pets and a pet holds many visits.

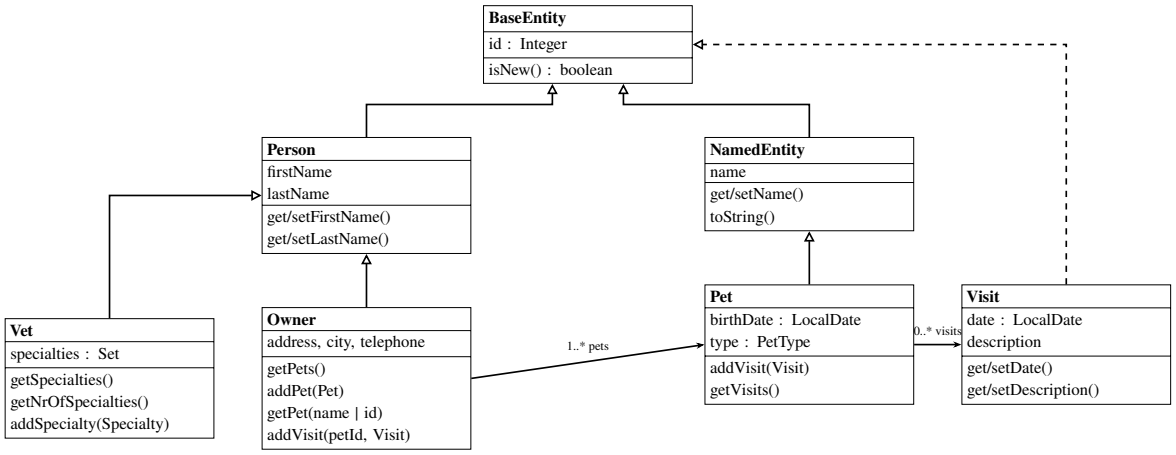



**Fig. 3.1.** Domain class diagram derived from the `model`, `owner` and `vet` packages (`PetType` and `Specialty` extend `NamedEntity`).

## 3.2 Architecture (MVC)

The application uses Spring MVC. Controllers handle HTTP requests, Spring Data repositories (`OwnerRepository`, `VetRepository`, `PetTypeRepository`) isolate persistence, Thymeleaf templates render views, and the JPA entities form the model.

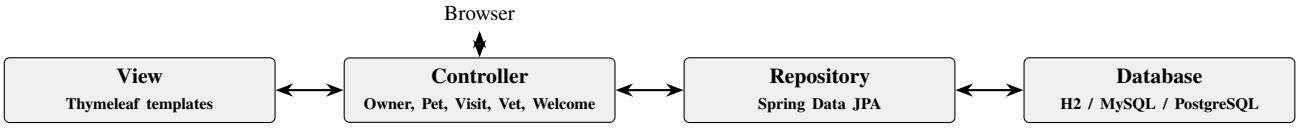



**Fig. 3.2.** Layered MVC architecture of PetClinic.

## 3.3 Interaction: Add a Visit (US4)

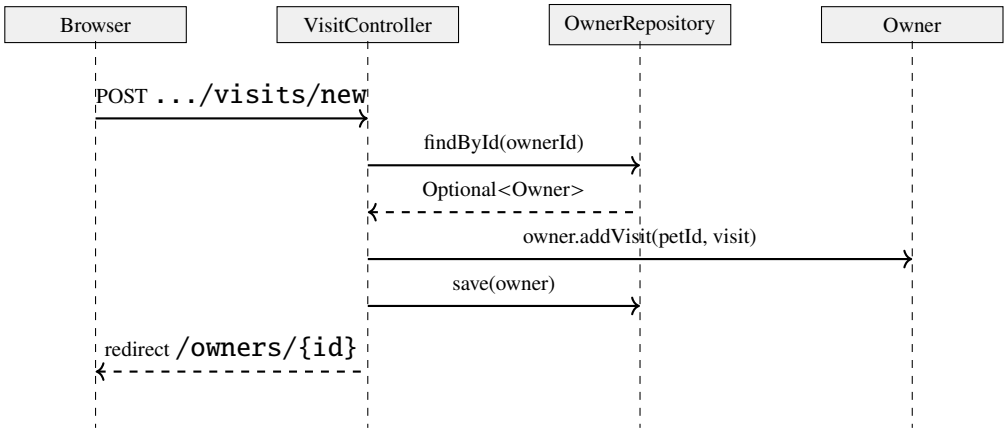



**Fig. 3.3.** Sequence diagram for recording a visit (validation errors re-render the form).

# CHAPTER 4 SOLID PRINCIPLES

| Principle | Evidence in the code |
|---|---|
| Single responsibility | Separate controllers per concern (`OwnerController`, `PetController`, `VisitController`, `VetController`); `PetValidator` only validates; `PetTypeFormatter` only converts. |
| Open/closed | Behaviour is extended through new Spring Data repository methods and formatters without modifying entity classes. |
| Liskov substitution | `Owner` and `Vet` are usable wherever a `Person` is expected; `Pet`, `PetType`, `Specialty` wherever a `NamedEntity` is. |
| Interface segregation | Small repository interfaces (`OwnerRepository`, `VetRepository`, `PetTypeRepository`) expose only what each controller needs. |
| Dependency inversion | Controllers depend on repository *interfaces*, injected by Spring; implementations are generated at runtime. |

**Table 4.1.** SOLID principles mapped to PetClinic.

# CHAPTER 5 TESTING AND VERIFICATION

## 5.1 Test Strategy

The suite mixes entity unit tests (`OwnerTests`, `VetTests`), validator tests, MVC slice tests (`OwnerControllerTests`, `PetControllerTests`, `VisitControllerTests`, `VetControllerTests`),

repository/service tests (`ClinicServiceTests`), integration tests (`PetClinicIntegrationTests`) and a concurrency test. Selected test cases mapped to the user stories:

| Story | Test class | Expected behaviour verified |
|---|---|---|
| US1 | `OwnerControllerTests` | Search by last name returns matching owners; unknown name shows none. |
| US2 | `OwnerControllerTests` | Valid owner is saved and redirected; blank fields return the form with errors. |
| US3 | `PetControllerTests`, `PetValidatorTests` | Pet with valid name/type/date is created; invalid input rejected. |
| US4 | `VisitControllerTests` | Visit is added and redirected; empty description rejected. |
| US5 | `VetControllerTests` | Vet list rendered with specialties. |
| US6 | `PetTests` | Age is whole years; birthday counted; null for unknown or future birth date. |

**Table 5.1.** Traceability from user stories to automated tests.

## 5.2 Execution Result

The suite was executed locally with `mvn test` (JDK 23 runtime, Maven build targeting Java 17):

**Result:** `Tests run: 85, Failures: 0, Errors: 0, Skipped: 2`
`BUILD SUCCESS`
79 upstream tests plus 6 new `PetTests`. The two skipped tests are the container-based MySQL/PostgreSQL integration tests, which need Docker.

## 5.3 Enhancement: Pet Age (US6)

Beyond analysing the upstream code, one small feature was added to show the construction workflow end to end (story → code → test → build). The owner page now shows each pet's age in whole years.

- **Model:** `Pet.getAgeInYears(LocalDate today)` returns whole years, or `null` when the birth date is unknown or in the future; a no-argument overload uses today's date. The `today` parameter keeps the logic testable without a clock.
- **View:** `ownerDetails.html` adds an Age row with the translated labels `age` and `years`, added to all ten `messages_*.properties` files so `I18nPropertiesSyncTest` keeps passing.
- **Tests:** six `PetTests` cases (whole years, unfinished year, birthday, newborn, unknown date, future date). Full suite after the change: 85 run, 0 failures.

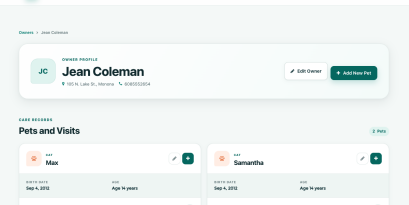



**Fig. 5.1.** Owner page after the enhancement: Max, born 2012-09-04, shows “Age 14 years”.

# CHAPTER 6 REPOSITORY AND CONFIGURATION MANAGEMENT

## 6.1 Repository Structure

The source is managed in Git (Apache License 2.0). The layout below was read from the cloned repository; templates, database scripts and tests sit beside the Java packages, which keeps design, code and test artefacts in one tree.

```
spring-petclinic/
  pom.xml, build.gradle, mvnw, gradlew build definitions and wrappers
  .github/workflows/ maven-build, gradle-build, deploy-and-test-cluster
  k8s/, docker-compose.yml deployment descriptors
  src/main/java/.../petclinic/
    model/ BaseEntity, NamedEntity, Person
    owner/ Owner, Pet, PetType, Visit + controllers, repositories, validator
    vet/ Vet, Specialty, Vets, VetController, VetRepository
    system/ WelcomeController, CrashController, CacheConfiguration, WebConfiguration
  src/main/resources/
    templates/ (owners, pets, vets, fragments) Thymeleaf views
    db/ (h2, mysql, postgres) schema.sql + data.sql per database
    messages/ i18n bundles
  src/test/java/.../petclinic/ 19 test classes mirroring the main packages
```

## 6.2 Naming Conventions and Configuration

Packages are organised by feature (`owner`, `vet`) rather than by technical layer; controllers end in `Controller`, Spring Data interfaces in `Repository`, tests in `Tests`. Database choice is configuration, not code: H2 by default, with `application-mysql.properties` and `application-postgres.properties` selected through Spring profiles, and a matching `schema.sql`/`data.sql` pair per database.

## 6.3 Data Model

The relational schema (`db/h2/schema.sql`) defines seven tables that map to the entity classes of Chapter 3.

| Table | Role |
|---|---|
| `owners` | Client details (name, address, city, telephone); maps to `Owner`. |
| `types` | Pet types such as cat and dog; maps to `PetType`. |
| `pets` | Pet name, birth date, type and owner; maps to `Pet`. |
| `visits` | Visit date and description for a pet; maps to `Visit`. |
| `vets` | Veterinarian names; maps to `Vet`. |
| `specialties` | Specialty names; maps to `Specialty`. |
| `vet_specialties` | Join table for the many-to-many link between vets and specialties. |

**Table 6.1.** Database tables and their entity classes.

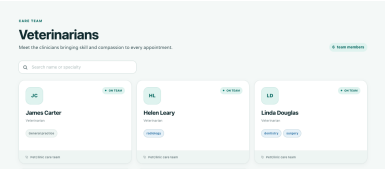



**Fig. 6.1.** Veterinarians page served from the `vets` and `specialties` tables.

# CHAPTER 7 CI/CD PIPELINE

The repository's own GitHub Actions workflow (`.github/workflows/maven-build.yml`) runs on every push and pull request to `main`: it checks out the code, sets up JDK 17 with Maven caching, and runs `./mvnw -B verify`. An equivalent Azure Pipelines definition for the course (written for this report, *not executed on Azure DevOps*) is below. Its stages are checkout and JDK setup, `verify` (compile, run all tests, package), and publishing test results so a failing test blocks the merge:

```
trigger: [ main ]
pool: { vmImage: ubuntu-latest }
steps:
  - task: JavaToolInstaller@0
    inputs: { versionSpec: '17', jdkArchitectureOption: x64, jdkSourceOption: PreInstalled }
  - script: ./mvnw -B verify
    displayName: Build and test
  - task: PublishTestResults@2
    inputs: { testResultsFormat: JUnit, testResultsFiles: '**/surefire-reports/TEST-*.xml' }
```

# CHAPTER 8 ENHANCEMENTS: REDESIGNED UI AND REST API

## 8.1 User Interface (branch `ui-redesign`)

The Thymeleaf templates were restyled with a CSS-variable design system, a remembered light/dark toggle, a responsive layout and no external CDN. New features: a home page with live owner/pet/vet/visit counts, an owner profile with initials avatar, pet cards and visit timeline (including the pet age from US6), and instant client-side filtering of the owner and vet lists.

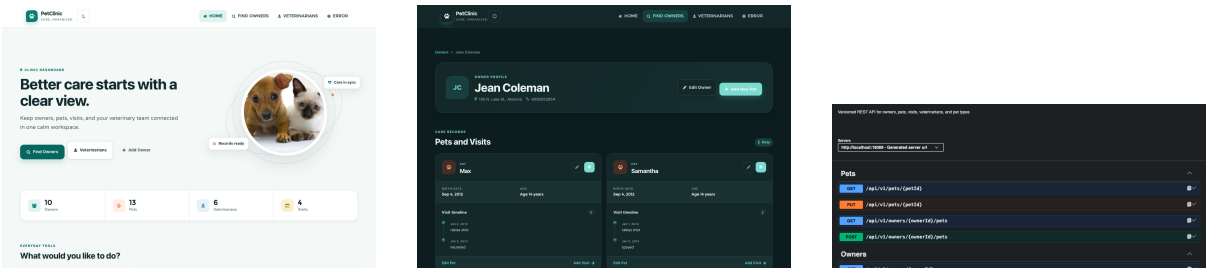



**Fig. 8.1.** Home (light), owner profile (dark) and Swagger UI from the running app.

## 8.2 REST API and Production Hardening (branch `api-backend`)

- **API v1:** `/api/v1/owners`, `pets`, `visits`, `vets`, `pet-types`; record DTOs, pagination and sorting, bean validation, RFC 7807 problem+json errors, proper status codes, and OpenAPI 3.1 docs with Swagger UI.
- **Observability:** actuator health (liveness/readiness), metrics and Prometheus; a correlation-id filter that logs each request.
- **Protection:** in-memory per-IP rate limit (429 with `Retry-After`), security headers including a Content-Security-Policy, and Caffeine caching with ETag on read endpoints.
- **Delivery:** non-root multi-stage Dockerfile, docker-compose with PostgreSQL, hardened GitHub Actions workflow.

Verified: `mvn test` gives `Tests run: 110, Failures: 0, Errors: 0, Skipped: 2` (85 original plus 25 new MockMvc tests). Live check: invalid POST gave a 400 problem document; 150 rapid requests gave 56 × 200, 94 × 429.

# CHAPTER 9 CONCLUSION

Spring PetClinic shows how a small codebase can still apply the full construction toolkit: clear layering, a compact domain model, SOLID-aligned class design, a test suite that maps to the user stories (110 tests, all passing locally: 85 including six for the pet-age feature, plus 25 for the REST API) and a CI workflow that rebuilds and re-tests every change. Azure Boards, Repos and Pipelines were not used, so no screens are shown for them.